03_lightgbm.ipynb

Training LightGBM for Web Traffic Time Series Forecasting

This notebook loads the engineered features from Notebook 02 and trains a LightGBM model to predict 60 future daily pageview values.

Steps:

- Load features and labels
- Prepare dataset
- Train LightGBM regressor
- Predict the next 60 days
- Compute SMAPE
- Save outputs to the Results folder

Imports

```
import pandas as pd
import numpy as np
import lightgbm as lgb
import matplotlib.pyplot as plt
from sklearn.metrics import mean_absolute_error
import os
```

Mount Google Drive

```
from google.colab import drive
drive.mount('/content/drive')
```

```
Mounted at /content/drive
```

Load engineered features and labels

From Notebook 02, we have two files:

- `features.csv` → Contains lag/rolling + metadata
- `labels.csv` → Contains the next 60 days we want to predict

```
# path based on where you saved your files
features_path = "/content/drive/MyDrive/CSC480/Final Project/features.csv"
labels_path = "/content/drive/MyDrive/CSC480/Final Project/labels.csv"

df_feat = pd.read_csv(features_path)
df_labels = pd.read_csv(labels_path)

df_feat.head(), df_labels.head()
```

```
   2015-07-03  2015-07-04  2015-07-05  2015-07-06  2015-07-07  2015-07-08  \
0    1.609438    1.945910    2.197225    1.945910    1.609438    0.000000
1    5.736572    5.429346    5.774552    5.743003    5.493061    5.468060
2    2.995732    3.433987    3.091042    3.218876    2.890372    5.187386
3    6.652863    7.060476    6.304449    6.628041    6.320768    6.204558
4    7.441320    7.358831    7.336286    7.363914    7.383368    7.457032

   2015-07-09  ...  2016-12-25_roll_std  2016-12-26_roll_std  \
0    1.098612  ...             0.506236             0.514896
1    5.497168  ...             0.169041             0.190357
2    3.713572  ...             0.259850             0.429010
3    8.476788  ...             0.196998             0.198785
4    7.560080  ...             0.046309             0.089099

   2016-12-27_roll_std  2016-12-28_roll_std  2016-12-29_roll_std  \
0             0.534076             0.270985             0.270985
1             0.196959             0.220090             0.214486
2             0.440246             0.421489             0.425851
3             0.233202             0.232868             0.233754
4             0.095800             0.098498             0.098561

   2016-12-30_roll_std  2016-12-31_roll_std  lang_enc  access_enc  agent_enc
0             0.270985             0.216804       248           2          0
1             0.230481             0.233897       236           0          1
2             0.494383             0.547694       337           2          0
3             0.232204             0.269482       341           0          1
4             0.097406             0.102299       223           2          1

[5 rows x 3307 columns],
   2016-11-02  2016-11-03  2016-11-04  2016-11-05  2016-11-06  2016-11-07  \
0    2.944439    1.945910    1.945910    1.609438    1.386294    2.484907
1    6.115892    6.061457    5.726848    5.852202    6.061457    5.910797
2    1.386294    1.945910    1.098612    1.609438    0.000000    2.564949
3    6.706862    6.558198    6.844815    6.315358    6.648985    6.626718
4    8.643650    8.463370    8.562167    8.631236    8.684909    8.499029

   2016-11-08  2016-11-09  2016-11-10  2016-11-11  ...  2016-12-22  \
0    2.197225    1.945910    1.791759    2.197225  ...    1.945910
1    5.823046    5.676754    5.886104    5.652489  ...    5.446737
2    1.791759    0.693147    1.098612    0.693147  ...    1.945910
3    6.410175    6.028279    6.109248    6.133398  ...    5.733341
4    8.216628    7.957177    7.928406    7.934155  ...    7.769801
```

```
   2016-12-29  2016-12-30  2016-12-31
0    1.945910    1.945910    2.302585
1    5.594711    5.308268    5.252273
2    1.609438    2.484907    2.564949
3    5.703782    5.726848    5.356586
4    7.765993    7.737180    7.675082

[5 rows x 60 columns])
```

Prepare LightGBM Inputs

LightGBM requires:

- X: numeric features
- y: regression targets

The target is a vector of length 60 for each sample.

```
# Features are all numeric except Page and metadata, already encoded
X = df_feat.select_dtypes(include=[np.number])

# Convert labels into numpy array
y = df_labels.values

print("X shape:", X.shape)
print("y shape:", y.shape)
```

```
X shape: (500, 3303)
y shape: (500, 60)
```

Train LightGBM

We train a regression model for each of the 60 forecast horizons.

This is because LightGBM does not natively support multi-output regression.

```
models = []
preds = []

params = {
    "objective": "regression",
    "metric": "l1",
    "learning_rate": 0.05,
    "num_leaves": 31,
    "verbose": -1
}

for i in range(60):
    print(f"Training horizon {i+1}/60")

    y_target = y[:, i]
```

```
    train_data = lgb.Dataset(X, label=y_target)

    model = lgb.train(params, train_data, num_boost_round=200)
    models.append(model)

    pred = model.predict(X)
    preds.append(pred)

# Convert predictions back to (rows x 60)
pred_matrix = np.column_stack(preds)
pred_matrix.shape
```

```
Training horizon 4/60
Training horizon 5/60
Training horizon 6/60
Training horizon 7/60
Training horizon 8/60
Training horizon 9/60
Training horizon 10/60
Training horizon 11/60
Training horizon 12/60
Training horizon 13/60
Training horizon 14/60
Training horizon 15/60
Training horizon 16/60
Training horizon 17/60
Training horizon 18/60
Training horizon 19/60
Training horizon 20/60
Training horizon 21/60
Training horizon 22/60
Training horizon 23/60
Training horizon 24/60
Training horizon 25/60
Training horizon 26/60
Training horizon 27/60
Training horizon 28/60
Training horizon 29/60
Training horizon 30/60
Training horizon 31/60
Training horizon 32/60
Training horizon 33/60
Training horizon 34/60
Training horizon 35/60
Training horizon 36/60
Training horizon 37/60
Training horizon 38/60
Training horizon 39/60
Training horizon 40/60
Training horizon 41/60
Training horizon 42/60
Training horizon 43/60
Training horizon 44/60
Training horizon 45/60
Training horizon 46/60
```

```
Training horizon 50/60
Training horizon 51/60
Training horizon 52/60
Training horizon 53/60
Training horizon 54/60
Training horizon 55/60
Training horizon 56/60
Training horizon 57/60
Training horizon 58/60
Training horizon 59/60
Training horizon 60/60
(500, 60)
```

Compute SMAPE (Symmetric Mean Absolute Percentage Error)

Competition Metric:

SMAPE = 2 * |forecast - actual| / (|forecast| + |actual| + epsilon)

```
def smape(true, pred):
    numer = np.abs(true - pred)
    denom = np.abs(true) + np.abs(pred) + 1e-8
    return np.mean(2 * numer / denom)

smape_score = smape(df_labels.values, pred_matrix)
print("LightGBM SMAPE:", smape_score)
```

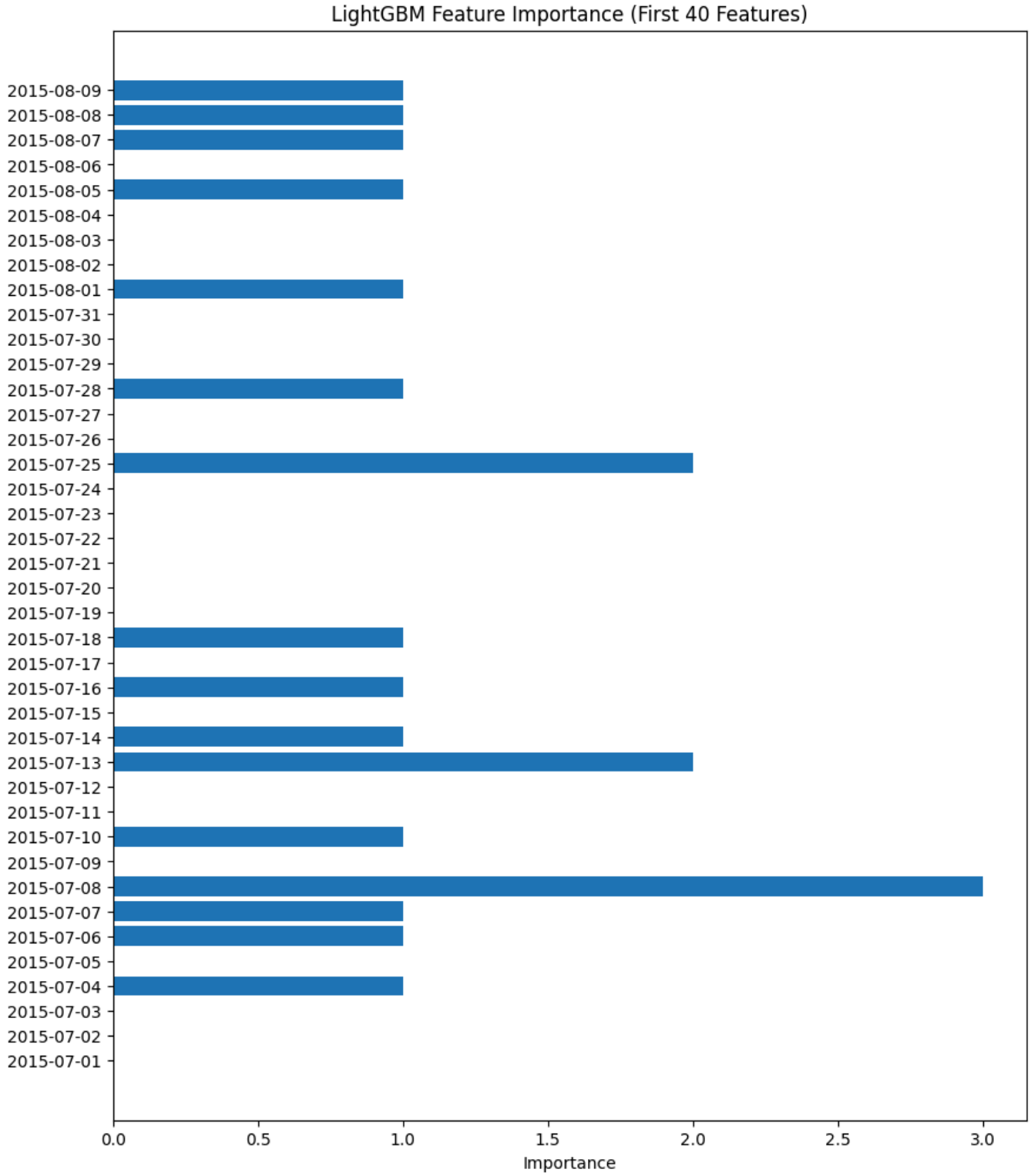
```
LightGBM SMAPE: 0.0695168151121957
```

Feature Importance (From Horizon 1 Model)

We display feature importances from the first LightGBM model.

```
model0 = models[0]
importance = model0.feature_importance()
features = X.columns

plt.figure(figsize=(10, 12))
plt.barh(features[:40], importance[:40])
plt.title("LightGBM Feature Importance (First 40 Features)")
plt.xlabel("Importance")
plt.show()
```


LightGBM Feature Importance (First 40 Features)
2015-08-09
2015-08-08
2015-08-07
2015-08-06
2015-08-05
2015-08-04
2015-08-03
2015-08-02
2015-08-01
2015-07-31
2015-07-30
2015-07-29
2015-07-28
2015-07-27
2015-07-26
2015-07-25
2015-07-24
2015-07-23
2015-07-22
2015-07-21
2015-07-20
2015-07-19
2015-07-18
2015-07-17
2015-07-16
2015-07-15
2015-07-14
2015-07-13
2015-07-12
2015-07-11
2015-07-10
2015-07-09
2015-07-08
2015-07-07
2015-07-06
2015-07-05
2015-07-04
2015-07-03
2015-07-02
2015-07-01
0.0
0.5
1.0
1.5
2.0
2.5
3.0
Importance

Save Model Predictions

We store:

- LightGBM predictions for 60 steps
- SMAPE score

This will be used later in the ensemble model (Notebook 05).

```
save_dir = "/content/drive/MyDrive/CSC480/Final Project/"
os.makedirs(save_dir, exist_ok=True)

# Save predictions matrix (rows x 60)
pred_path = save_dir + "lightgbm_predictions.csv"
np.savetxt(pred_path, pred_matrix, delimiter=",")

# Save SMAPE score
score_path = save_dir + "lightgbm_smape.txt"
with open(score_path, "w") as f:
  f.write(str(smape_score))

print("Saved LightGBM predictions to:", pred_path)
print("Saved LightGBM SMAPE score to:", score_path)
```

```
Saved LightGBM predictions to: /content/drive/MyDrive/CSC480/Final Project/lightgb
Saved LightGBM SMAPE score to: /content/drive/MyDrive/CSC480/Final Project/lightgb
```